Homework #1

Vanja Stojanović

March 2026

1 Connected components

We assume a graph G s.t. $|V(G)| = n$ and $|E(G)| = m$, and it contains c connected components. We prove the following:

Proposition 1.

$$n - c \leq m \leq \binom{n-c+1}{2}$$

Proof. We proceed with induction on n for the first inequality.
Base case (n = 1). It trivially follows that $c = 1$ and $m = 0$, hence vacuously $1 - 1 = 0 \leq 0$.
Induction hypothesis (IH). We assume the relation holds for n.
Induction step. Assume for a graph G with $V(G) = n$.

Let G' be the graph we obtain by adding a vertex v to G. By the IH our inequality holds for G. When we add a vertex v to G, we can arbitrarily choose its degree in the interval $0 \leq \deg(v) \leq n$, hence we increase the number of vertices to $n' = n + 1$, the number of connected components can stay the same or increase by one if $\deg(v) = 0$ and we increase the number of edges to $m' = m + \deg(v)$.
If $\deg(v) \geq 1$, then c' in G' can only decrease or stay the same. If v connects to k distinct components (where $k \leq \deg(v)$), then $c' = c - k + 1$:

$$(n+1) - (c-k+1) \leq m + \deg(v) \tag{1}$$
$$n - c + k \leq m + \deg(v) \tag{2}$$

which holds by the IH and the fact that $k \leq \deg(v)$.
If $\deg(v) = 0$, then

$$(n+1) - (c+1) \leq m + 0 \tag{3}$$
$$n - c \leq m \tag{4}$$

which again holds by the IH, proving our induction step.

For the second inequality, let n_i denote the number of vertices in component i. Then $n = \sum_{i=1}^{c} n_i$ with $n_i \geq 1$, and each component has at most $\binom{n_i}{2}$ edges, so

$$m \leq \sum_{i=1}^{c} \binom{n_i}{2}.$$

We claim this sum is maximized when one component has $n-c+1$ vertices and the remaining $c-1$ are singletons. To see this, suppose two components have $n_a \geq 2$ and $n_b \geq 2$. Moving a vertex from b to a changes the sum by

$$\binom{n_a+1}{2} + \binom{n_b-1}{2} - \binom{n_a}{2} - \binom{n_b}{2} = n_a - (n_b - 1) = n_a - n_b + 1.$$

If $n_a \geq n_b$, this is positive, so the sum strictly increases. Repeating this process, we concentrate all non-singleton vertices into one component, giving

$$m \leq \binom{n-c+1}{2}.$$

□

By Harary's theorem [1], a disconnected graph on n vertices has at most $\binom{n-1}{2}$ edges. This follows directly from our upper bound with $c \geq 2$:

Corollary 2. If $m > \binom{n-1}{2}$, then G is connected.

Proof. By contrapositive. If G is disconnected ($c \geq 2$), then by Proposition 1, $m \leq \binom{n-c+1}{2} \leq \binom{n-1}{2}$. □

Remark 3. This criterion is not practically useful for real networks. Most real-world graphs, like social networks [2], internet topology, and biological networks are sparse, with $m = O(n)$ or $O(n \log n)$, yet are typically connected. The threshold $\binom{n-1}{2} = O(n^2)$ is far too conservative. For instance, Erdős and Rényi [4] showed that the sharp threshold for connectivity in the random graph $G(n,p)$ is $p = \frac{\ln n}{n}$, corresponding to $m \approx \frac{n \ln n}{2}$, which is much smaller than $O(n^2)$.

2 Node linking probability

We consider such a random graph G, where an edge between $i \in V(G)$ and $j \in V(G)$ is present with probability p_{ij}. The probability is proportional to p_{ij} $v_i v_j$, where $v_i \geq 0$ is some number associated with node i.

Proposition 4. The expected node degree $\mathbb{E}(k_i)$ of a node i is proportional to v_i, where $k_i = deg(i)$.

Proof. Since each edge is a Bernoulli trial, we can express the expected value of the node degree of $i \in V(G)$ as

$$\mathbb{E}(k_i) = \sum_{j \neq i} p_{ij} = \sum_{j \neq i} \alpha v_i v_j = \alpha v_i \sum_{j \neq i} v_j$$

For large networks, any single v_i is negligible compared to the total $S = \sum_j v_j$, so $\sum_{j \neq i} v_j = S - v_i \approx S$. Hence $\mathbb{E}(k_i) \approx C v_i$, where $C = \alpha S$ is a constant independent of i, and α is the proportionality constant. □

We can now express p_{ij} in terms of the expected node degree.

Proposition 5. We can write the probability p_{ij} in terms of the expected vertex degree of vertices i and j.

Proof. By Proposition 4 we have $\mathbb{E}(k_i) = v_i\big(\alpha \sum_{j \neq i} v_j\big)$. We substitute it into the relation for p_{ij}

$$\begin{aligned}
p_{ij} &= \alpha \cdot \frac{\mathbb{E}(k_i)}{\alpha \sum_j v_j} \cdot \frac{\mathbb{E}(k_j)}{\alpha \sum_j v_j} \\
&= \frac{\mathbb{E}(k_i)\mathbb{E}(k_j)}{\alpha \big(\sum_j v_j\big)^2} \\
&= \frac{\mathbb{E}(k_i)\mathbb{E}(k_j)}{\alpha \Big(\sum_j v_j\Big)\Big(\sum_l v_l\Big)} \\
&= \frac{\mathbb{E}(k_i)\mathbb{E}(k_j)}{\sum_l \alpha v_l \Big(\sum_j v_j\Big)} \\
&= \frac{\mathbb{E}(k_i)\mathbb{E}(k_j)}{\sum_l \mathbb{E}(k_l)}
\end{aligned}$$

where we use Proposition 4 in the last step. □

The derivation in the proof of Proposition 5 is formally known as the Chung-Lu model [3].

3 Random node selection

Assume for a graph G. We design an algorithm which takes in a graph (network) data structure representation, and chooses a random node (vertex) in $O(1)$, where the probability of a node being chosen is $p = \frac{deg(i)}{2m}$ where $i \in V(G)$.

Algorithm 1 Degree-proportional node selection

Require: Edge list $E = [e_1, e_2, \ldots, e_{2m}]$ containing both endpoints of each edge

```
idx ← UniformRandom(1, 2m)
return E[idx]
```

Theorem 6. Algorithm 3 selects a node with probability $P(i) = \frac{deg(i)}{2m}$.

Proof. Assume E is the edge list inputted into the algorithm. The edge list is constructed out of both endpoints of all edges, since each edge has 2 endpoints, the number of elements in the list is $2m$. Hence, when picking a node uniformly at random from the list, we can write the following equation

$$P(i) = \frac{|\{x : x \in E \wedge x = i\}|}{2m}$$

since each vertex will appear exactly as many times in the list as is the number of its edges

$$P(i) = \frac{deg(i)}{2m}$$

□

4 Weak & strong connectivity

For this section, we assume for a digraph D. If we were to apply the depth-first search (DFS hereon) to D and take into account the three given criteria, the result would be:

1. **If we follow the links as if the ordered pair would be unordered**. Then the algorithm would be equivalent as if we took the unoriented graph G of D and applied the DFS on it. Hence we would get weakly connected components of D.

2. **If we follow the links in the direction of the ordered pair**. We get the "visibility" graph of some vertex, i.e. which vertices we can reach starting at some vertex.

3. **If we follow the links in the opposite direction of the ordered pair**. We get the "reverse-visibility" graph of some vertex, i.e. which vertices can reach that vertex.

From answers 2 and 3, vertex u is in the same SCC as v iff u is reachable from v following directed edges, and v is reachable from u. We use this to find all SCCs:

Algorithm 2 Strongly connected components via forward and backward DFS

Require: Digraph $D = (V, E)$
Ensure: Set of all SCCs

```
unvisited ← V
SCCs ← ∅
while unvisited ≠ ∅ do
    Pick any v ∈ unvisited
    F ← DFS(v, D)                    ▷ vertices reachable from v
    B ← DFS(v, D^R)                  ▷ vertices that can reach v
    SCC(v) ← F ∩ B
    SCCs ← SCCs ∪ {SCC(v)}
    unvisited ← unvisited \ SCC(v)
end while
return SCCs
```

Implementing the algorithm and running it on the `enron.net` network, we get that the network has 78,058 strongly connected components, and the larget one has 9,164 of vertices. This is expected for a communication network like Enron's email graph. Most vertices are in small SCCs (many singletons — people who only sent or only received emails), while there's one large SCC forming the core of bidirectional communication.

5 Is Java software scale-free?

We analyze the Lucene search engine class dependency network [5] with $n = 2956$ vertices and $m = 10872$ arcs. Figure 1 shows the degree distributions on a log-log plot.

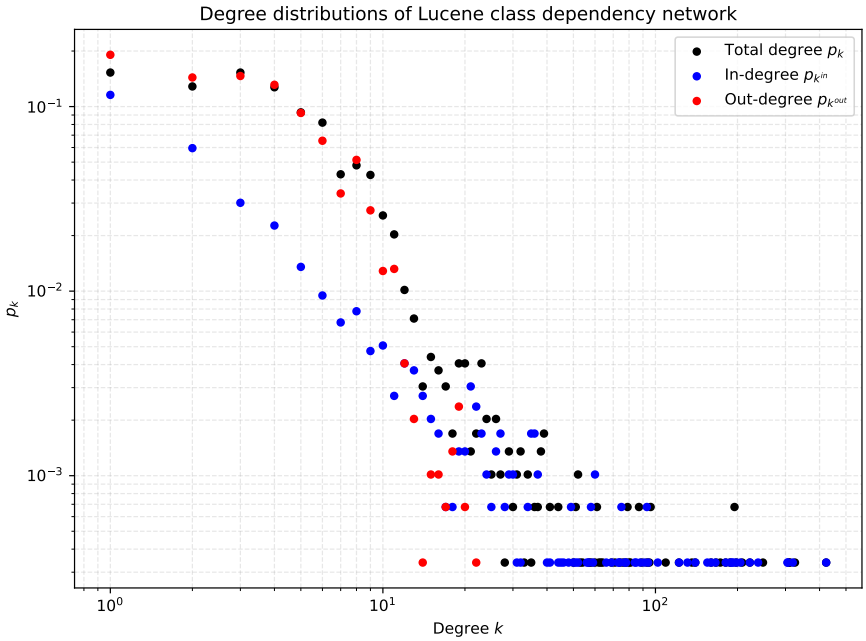

Figure 1: Degree distributions of the Lucene class dependency network.

Using the MLE formula from [6] with $k^{\cdot}_{\min} = 5$, computed separately for in- and out-degree:

$$\gamma_{\cdot} = 1 + n_{\cdot} \left(\sum_{i=1}^{n} \ln \frac{k^{\cdot}_i}{k^{\cdot}_{\min} - \frac{1}{2}} \cdot \delta(k^{\cdot}_i \geq k^{\cdot}_{\min}) \right)^{-1}$$

where $\delta(k^{\cdot}_i \geq k^{\cdot}_{\min})$ is the indicator function and $n_{\cdot} \leq n$ is the number of nodes with $k^{\cdot}_i \geq k^{\cdot}_{\min}$. We obtain:

- **In-degree**: appears scale-free, roughly linear on the log-log plot. $\gamma_{\text{in}} \approx 1.80$ ($n_{\text{in}} = 315$). This is consistent with preferential attachment [2]: widely-used utility classes (interfaces, base classes) accumulate dependencies from many other classes.

- **Out-degree**: appears scale-free, though with more curvature than in-degree. $\gamma_{\text{out}} \approx 3.37$ ($n_{\text{out}} = 917$). The steeper exponent reflects that out-degree

(how many dependencies a class has) is more constrained by software design principles such as limited coupling.

- **Total degree**: appears scale-free as a mixture of both distributions. $\gamma_{\text{total}} \approx 2.47$ ($n_{\text{total}} = 1296$), sitting between in- and out-degree on the log-log plot.

The code used to produce the plot and compute the exponents:

```
import numpy as np
import matplotlib.pyplot as plt
from collections import defaultdict, Counter


def parse_net(path):
    in_degree = defaultdict(int)
    out_degree = defaultdict(int)
    vertices = set()

    with open(path) as f:
        section = None
        for line in f:
            line = line.strip()
            if line.startswith("*vertices"):
                section = "vertices"
                continue
            elif line.startswith("*arcs"):
                section = "arcs"
                continue

            if section == "vertices":
                vid = int(line.split()[0])
                vertices.add(vid)
                in_degree[vid] = in_degree.get(vid, 0)
                out_degree[vid] = out_degree.get(vid, 0)
            elif section == "arcs":
                parts = line.split()
                u, v = int(parts[0]), int(parts[1])
                out_degree[u] += 1
                in_degree[v] += 1

    return vertices, in_degree, out_degree


def degree_distribution(degree_dict):
    counts = Counter(degree_dict.values())
    n = len(degree_dict)
    ks = sorted(k for k in counts if k > 0)
    pk = {k: counts[k] / n for k in ks}
    return ks, pk
```

```python


def mle_gamma(degrees, k_min):
    filtered = [k for k in degrees if k >= k_min]
    n = len(filtered)
    if n == 0:
        return None, 0
    gamma = 1 + n * (sum(np.log(k / (k_min - 0.5)) for k in filtered)) ** -1
    return gamma, n


def plot_distributions(total_ks, total_pk, in_ks, in_pk, out_ks, out_pk):
    fig, ax = plt.subplots(figsize=(8, 6))

    ax.scatter(
        total_ks,
        [total_pk[k] for k in total_ks],
        c="black",
        s=20,
        label="Total degree $p_k$",
        zorder=3,
    )
    ax.scatter(
        in_ks,
        [in_pk[k] for k in in_ks],
        c="blue",
        s=20,
        label="In-degree $p_{k^{in}}$",
        zorder=3,
    )
    ax.scatter(
        out_ks,
        [out_pk[k] for k in out_ks],
        c="red",
        s=20,
        label="Out-degree $p_{k^{out}}$",
        zorder=3,
    )

    ax.set_xscale("log")
    ax.set_yscale("log")
    ax.set_xlabel("Degree $k$")
    ax.set_ylabel("$p_k$")
    ax.set_title("Degree distributions of Lucene class dependency network")
    ax.legend()
    ax.grid(True, which="both", ls="--", alpha=0.3)
```

```python

    plt.tight_layout()
    plt.savefig("HW1/build/lucene_degree_distributions.pdf")
    plt.savefig("HW1/build/lucene_degree_distributions.png", dpi=150)
    print("Plot saved to HW1/build/lucene_degree_distributions.pdf")


if __name__ == "__main__":
    vertices, in_deg, out_deg = parse_net("HW1/networks/lucene.net")
    total_deg = {v: in_deg[v] + out_deg[v] for v in vertices}

    total_ks, total_pk = degree_distribution(total_deg)
    in_ks, in_pk = degree_distribution(in_deg)
    out_ks, out_pk = degree_distribution(out_deg)

    plot_distributions(total_ks, total_pk, in_ks, in_pk, out_ks, out_pk)

    print("\nMLE power-law exponents")
    k_min = 5
    for name, deg_dict in [("Total", total_deg), ("In", in_deg), ("Out", out_deg)]:
        degrees = [k for k in deg_dict.values() if k > 0]
        gamma, n_used = mle_gamma(degrees, k_min)
        if gamma:
            print(f"{name}-degree: gamma = {gamma:.3f} (k_min = {k_min}, n = {n_used})")
```

6 Five networks problem

We are given five networks in edge list format and must identify each by examining structural properties. Two of the networks are directed. We compute the following metrics for each network: number of nodes n, number of edges/arcs, average degree $\mathbb{E}(k)$, maximum degree, median degree, clustering coefficient C (estimated via random sampling of 5000 nodes), and directedness (by comparing arc count to undirected edge count, if arcs > undirected edges, the graph contains reciprocated directed edges).

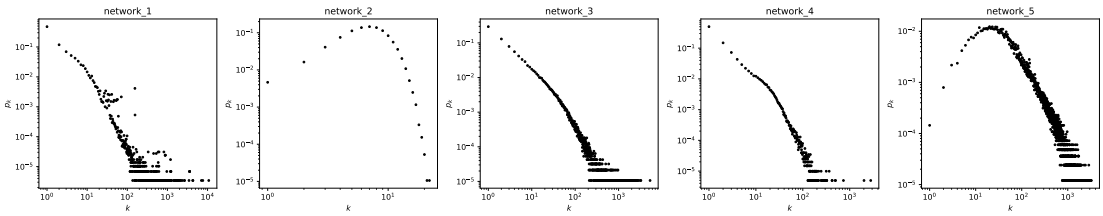

Figure 2: Degree distributions of the five networks on log-log scale.

Network	n	lines	m (undir.)	$\mathbb{E}(k)$	$k_{\max}$	$\tilde{k}$	C	recip.
`network_1`	291,934	1,399,248	1,037,011	7.10	10,721	2	0.49	0.259
`network_2`	189,207	703,484	703,484	7.44	22	7	0.00	0.000
`network_3`	94,093	766,258	766,258	16.29	5,311	3	0.00	0.000
`network_4`	202,275	510,180	509,097	5.03	2,803	1	0.16	0.002
`network_5`	83,159	5,510,251	5,510,251	132.52	3,257	58	0.62	0.000

Table 1: Structural properties of the five networks. $\tilde{k}$ denotes median degree.

Step 1: Identify directed networks. Comparing arc count (lines) to undirected edge count: `network_1` has 1,399,248 arcs but only 1,037,011 undirected edges, indicating directed edges with 25.9% reciprocity. `network_4` has 510,180 arcs vs. 509,097 undirected edges, a small but nonzero difference (1,083 reciprocated arcs, 0.2% reciprocity), indicating a directed network. The remaining three are undirected.

Step 2: Identify the Erdős-Rényi graph. `network_2` has a maximum degree of only 22 across 189,207 nodes, a median of 7, and zero clustering. The degree distribution (Figure 2) shows a narrow bell shape on the log-log plot, consistent with a Poisson distribution rather than a power law. This matches the $G(n, p)$ model [4].

$\Rightarrow$ `network_2` = **Erdős-Rényi random graph.**

Step 3: Identify the Flickr network. `network_5` is undirected with an extremely high average degree ($\langle k \rangle = 132.5$), the highest clustering coefficient ($C =$

0.62), and a power-law degree distribution. Social affiliation networks are characteristically dense with high transitivity [2, 7].

$\Rightarrow$ `network_5` = **Flickr social affiliation network.**

Step 4: Identify the Web graph. `network_1` is directed with moderate reciprocity (25.9%, web pages often mutually link). It has high clustering ($C = 0.49$) and a power-law distribution with extreme hubs ($k_{\max} = 10{,}721$), consistent with popular hub pages in a university web graph [6].

$\Rightarrow$ `network_1` = **University Web graph.**

Step 5: Identify the citation network. `network_4` is directed with near-zero reciprocity (0.2%), which is expected since citations are almost never mutual [7]. The median degree of 1 indicates many papers with very few citations. It has moderate clustering ($C = 0.16$) from co-citation patterns and a power-law tail ($k_{\max} = 2{,}803$) from a few highly-cited papers [6].

$\Rightarrow$ `network_4` = **CS citation network.**

Step 6: Identify the IMDb network. `network_3` is the remaining undirected network. It has a power-law degree distribution ($k_{\max} = 5{,}311$, prolific actors), average degree of 16.29, and zero clustering. The zero clustering suggests a bipartite actor-movie representation rather than a projected actor-actor graph, as bipartite graphs contain no odd cycles and thus no triangles [7].

$\Rightarrow$ `network_3` = **IMDb actors collaboration network.**

The code used to compute the structural properties:

```
import random
import numpy as np
import matplotlib.pyplot as plt
from collections import defaultdict, Counter


def parse_adj(path):
    adj = defaultdict(set)
    directed_edges = set()
    n_header = 0
    m_header = 0
    line_count = 0

    with open(path) as f:
        for line in f:
            line = line.strip()
            if line.startswith("#"):
                if "nodes" in line and "edges" in line:
                    parts = line.split()
                    for i, p in enumerate(parts):
```

```
                        if p == "nodes":
                            n_header = int(parts[i - 1].replace(",", ""))
                        if p == "edges":
                            m_header = int(parts[i - 1].replace(",", ""))
                continue
            if not line:
                continue
            parts = line.split()
            u, v = int(parts[0]), int(parts[1])
            adj[u].add(v)
            adj[v].add(u)
            directed_edges.add((u, v))
            line_count += 1

    vertices = set(adj.keys())
    n = len(vertices)
    degrees = [len(adj[v]) for v in vertices]
    m_undirected = sum(degrees) // 2

    # directed if line count > undirected edge count
    is_directed = line_count > m_undirected * 1.01
    reciprocity = (line_count - m_undirected) / line_count if line_count > 0 else 0

    return {
        "n": n,
        "m_undirected": m_undirected,
        "m_header": m_header,
        "line_count": line_count,
        "degrees": degrees,
        "is_directed": is_directed,
        "reciprocity": reciprocity,
        "adj": adj,
    }


def compute_stats(info):
    degrees = info["degrees"]
    n = info["n"]
    m = info["m_undirected"]

    avg_deg = np.mean(degrees)
    max_deg = np.max(degrees)
    med_deg = np.median(degrees)
    density = 2 * m / (n * (n - 1))

    deg_counts = Counter(degrees)
```

```python
    ks = sorted(k for k in deg_counts if k > 0)
    pk = {k: deg_counts[k] / n for k in ks}

    # clustering coefficient (random sample)
    adj = info["adj"]
    all_nodes = list(adj.keys())
    random.seed(42)
    sample_nodes = random.sample(all_nodes, min(5000, len(all_nodes)))
    cc_values = []
    for v in sample_nodes:
        neighbors = list(adj[v])
        k = len(neighbors)
        if k < 2:
            continue
        if k > 500:
            # sample pairs for high-degree nodes
            pairs = 0
            triangles = 0
            sampled_pairs = min(5000, k * (k - 1) // 2)
            for _ in range(sampled_pairs):
                a, b = random.sample(neighbors, 2)
                pairs += 1
                if b in adj[a]:
                    triangles += 1
            cc_values.append(triangles / pairs if pairs > 0 else 0)
        else:
            triangles = 0
            for i in range(len(neighbors)):
                for j in range(i + 1, len(neighbors)):
                    if neighbors[j] in adj[neighbors[i]]:
                        triangles += 1
            cc_values.append(2 * triangles / (k * (k - 1)))
    avg_cc = np.mean(cc_values) if cc_values else 0

    return {
        "avg_deg": avg_deg,
        "max_deg": max_deg,
        "med_deg": med_deg,
        "density": density,
        "avg_cc": avg_cc,
        "ks": ks,
        "pk": pk,
    }


if __name__ == "__main__":
    networks = {}
```

```python
    for i in range(1, 6):
        path = f"HW1/networks/network_{i}.adj"
        print(f"Parsing network_{i}...")
        info = parse_adj(path)
        stats = compute_stats(info)
        networks[i] = {**info, **stats}

    print(f"\n{'Net':>4} {'n':>10} {'lines':>12} {'m_undir':>12} {'<k>':>8} "
          f"{'max_k':>8} {'med_k':>8} {'density':>12} {'avg_cc':>8} {'directed':>10} {'recip':>8}")
    print("-" * 115)
    for i in range(1, 6):
        net = networks[i]
        print(f"{i:>4} {net['n']:>10,} {net['line_count']:>12,} {net['m_undirected']:>12,} "
              f"{net['avg_deg']:>8.2f} {net['max_deg']:>8} {net['med_deg']:>8.1f} "
              f"{net['density']:>12.2e} {net['avg_cc']:>8.4f} {str(net['is_directed']):>10} "
              f"{net['reciprocity']:>8.3f}")

    # degree distribution plots
    fig, axes = plt.subplots(1, 5, figsize=(20, 4))
    for i in range(1, 6):
        ax = axes[i - 1]
        net = networks[i]
        ax.scatter(net["ks"], [net["pk"][k] for k in net["ks"]], s=5, c="black")
        ax.set_xscale("log")
        ax.set_yscale("log")
        ax.set_title(f"network_{i}")
        ax.set_xlabel("$k$")
        ax.set_ylabel("$p_k$")
    plt.tight_layout()
    plt.savefig("HW1/build/five_networks_degree_dist.pdf")
    plt.savefig("HW1/build/five_networks_degree_dist.png", dpi=150)
    print("\nPlot saved to HW1/build/five_networks_degree_dist.pdf")
```

7 Who to vaccinate?

The second scheme (vaccinate a random acquaintance) is strictly better.
This is due to the following: selecting a node uniformly at random yields an expected spreading potential proportional to $\mathbb{E}(k^2)$. In the second scheme, following a random edge from a random node reaches a node of degree k with probability proportional to $k \cdot p_k$ rather than p_k, which is degree-biased sampling. The expected spreading potential of the vaccinated acquaintance is thus proportional to $\mathbb{E}(k^3)/\mathbb{E}(k)$, which exceeds $\mathbb{E}(k^2)$ for any non-degenerate degree distribution. This is an application of the *friendship paradox* [7]: a random neighbor of a random node has, on average, higher degree than a random node itself. Since spreading probability is proportional to k^2, the second scheme systematically removes higher-risk nodes from the network, providing more effective immunization with the same number of vaccines.

References

[1] F. Harary, *Graph Theory*, Addison-Wesley, 1969.

[2] A.-L. Barabási and R. Albert, Emergence of scaling in random networks, *Science*, 286(5439):509–512, 1999.

[3] F. Chung and L. Lu, Connected components in random graphs with given expected degree sequences, *Annals of Combinatorics*, 6:125–145, 2002.

[4] P. Erdős and A. Rényi, On random graphs I, *Publicationes Mathematicae Debrecen*, 6:290–297, 1959.

[5] L. Šubelj and M. Bajec, Community structure of complex software systems: Analysis and applications, *Physica A*, 390(16):2968–2975, 2011.

[6] A.-L. Barabási, *Network Science*, Cambridge University Press, 2016.

[7] M. E. J. Newman, *Networks: An Introduction*, Oxford University Press, 2010.

Homework #1

The homework does not require a lot of writing but may require some thinking. It does not require a lot of processing power but may require efficient programming. It accounts for 12.5% of the course grade. Any questions or comments should be posted on Piazza.

Submission details

This homework is due on **March 27th** at 11:59pm. It must be submitted through (1) Gradescope (entry code **RVYW5D**) and (2) eUcilnica. (1) Submission to Gradescope should include answers to all questions, each on a separate page, which may also demand pseudocode, proofs, tables, plots, diagrams and/or other. (2) Submission to eUcilnica should include this cover sheet with signed honor code and all the programming code used to complete the exercises (preferably in `.py` format). The homework is considered submitted only when *both* (1) and (2) have been submitted. Failing to include the honor code in the submission will result in **10% deduction**. Failing to submit all the developed code will result in **10% deduction**.

Honor code

Students are strongly encouraged to discuss the homework with other classmates and form study groups. However, each student must then solve the homework by herself/himself without the help of others and should be able to redo the homework at a later time. In other words, students are encouraged to collaborate but should not copy from one another. Referring to any solutions obtained from classmates, course books, previous years, found online, AI tools or other is considered an honor code violation. Also, stating any part of the solutions in class or on Piazza is considered an honor code violation.

Any violation of the honor code will be reported to the **faculty disciplinary committee** and vice dean for education.

SID: _____

Full name: _____

Study group: _____

I understand and accept the honor code.

Signature: _____